
Mini-VLM: A Pure RWKV Vision-Language Model for Efficient Visual Question Answering

Oliver Petrovic

1008923042, oliver.petrovic@mail.utoronto.ca

<https://github.com/Olipet24/mini-vlm>

Introduction

Modern Vision-Language Models (VLMs) need billions of parameters and heavy GPU resources, which puts them out of reach for everyday or mobile hardware. Mini-VLM sets out to answer a narrower question: how much of that capability can survive a strict **100MB storage budget**? Given a 320×320 image and a free-form text question, our model outputs one of the 1000 most common VQA answers (Figure 1). Mapping pixels and free-form text onto a correct answer needs learned, non-linear representations that hand-coded rules cannot approximate (Goyal et al., 2017). Rather than shrinking a Transformer-based VLM after the fact, our core contribution is a *unified RWKV pipeline*: an RWKV spatial bridge and language core replace attention everywhere, giving linear-time (not quadratic) cost in the combined visual+text sequence length (Peng et al., 2023), instead of the linear-projector-into-attention approach used by connectors such as LLaVA (Liu et al., 2023). We compare this directly against a control baseline that keeps the same frozen encoder but restores standard self-attention, and evaluate both on the full VQA v2.0 pool, a held-out test split, and newly collected photographs never used in tuning. The result: a sub-14MB model that answers real visual questions well above chance, matches a Transformer of comparable size on raw accuracy, and – because its cost stays fixed regardless of image resolution – is cheap enough to run three times over for a decisive ensemble win.

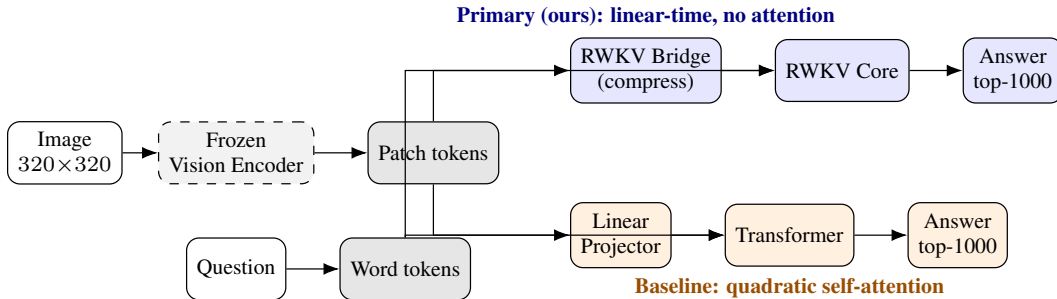


Figure 1: Both models share a frozen, offline-cached vision encoder and tokenizer; only the boxed bridge/projector and core/Transformer stages are trained. Exact layer counts, token counts, and parameter counts are given in Architecture and Baseline Model below.

Background & Related Work

Mini-VLM sits at the intersection of three lines of work. **Vision→language connectors.** LLaVA (Liu et al., 2023) projects every patch token through one linear layer straight into the language model, so sequence length – and attention cost – grows with image resolution. Flamingo’s Perceiver Resampler (Alayrac et al., 2022) instead learns a small fixed set of query tokens that cross-attend into the visual features, compressing a variable-size feature map to a constant-size summary; our RWKV Spatial Bridge shares this compression goal but replaces the cross-attention itself with a learned linear pooling matrix, avoiding attention entirely. BLIP-2’s Q-Former (Li et al., 2023) takes the cross-attention route further with a dedicated pretrained querying transformer – exactly the mechanism we avoid on compute-cost and parameter-budget grounds. **Efficient sequence models.** RWKV (Peng et al., 2023) replaces quadratic self-attention with a linear-time weighted-key-value recurrence; we use it as both the bridge and the language backbone, not just an efficient text decoder. **Compact deployment and VQA itself.** MobileNetV3 (Howard et al., 2019) targets exactly the mobile-inference regime we aim for and is our frozen backbone; GPTQ (Frantar et al., 2023) motivates our planned weight-compression path for the bridge/core’s Linear layers. VQA v2.0 (Goyal et al., 2017) was built

to reduce language-prior bias in the original VQA dataset, but Agrawal et al. (2018) show such priors persist – motivating the question-only ablation below (37.94% accuracy from vision-blind inputs), which quantifies how much of our accuracy really comes from the image. **Classic (non-web-pretrained) VQA SOTA.** Bottom-up-top-down attention (Anderson et al., 2018), MCAN (Yu et al., 2019), and BAN (Kim et al., 2018) reach 66–73% on VQA v2.0 alone – a different weight class, not a like-for-like comparison. They pair co-attention (which we deliberately avoid) with *bottom-up region features* from a separately-trained Faster R-CNN detector (~ 44 M params alone, larger than our entire model), and Pythia (Jiang et al., 2018) traces roughly half their further gain to *fine-tuning* those features end-to-end – exactly what our frozen, offline-cached, sub-100MB design forgoes by design. They also use VQA’s official soft accuracy (partial credit when only some of 10 annotators agree), while we score stricter top-1 exact-match on the same predictions – one more reason these numbers describe a differently-scoped system, not a shortfall on our part.

Data Processing

We use the official **VQA v2.0** dataset (questions/annotations over COCO images), cleaned in two steps to fit the 100MB budget: (1) collapse free-form answers to a **top-1000 most-common-answer classification** problem, built from the frequency distribution over the full 443,757-example train2014 pool (87.47% coverage), dropping examples outside that vocabulary; (2) resize/normalize images to 320×320 (upgraded from an initial 224×224 pass after finding resolution was one of our biggest accuracy levers, Discussion) and pass them through the frozen vision encoder *once, offline*, caching the resulting $576 \times 10 \times 10$ FP16 feature map so training never re-runs the CNN. train/val are disjoint samples from the official *train2014* pool; test is sampled entirely from *val2014* – images and questions never seen during training or hyperparameter selection: 368,751 train, 19,407 val, 186,489 test questions over 82,575 (train+val) / 40,404 (test) unique images, 122,979 cached feature tensors, answer-type mix 43.1% yes/no : 43.5% other : 13.4% number, mean question length 6.1 words. One cleaned example: image of a bird, question “What color is this bird?”, target class `white` and `brown` – one of the top-1000 classes covering 87.47% of the train2014 pool. Downloading and caching the full $\sim 123,000$ -image pool once was more reliable than smaller-scale per-image fetches; we also added a feature-integrity check that now runs before every training job, after catching one batch of silently corrupted cached tensors early via a training-loss spike.

Architecture

The primary model’s frozen **vision encoder** is MobileNetV3-Small, compressed FP32 $\rightarrow$ FP16 (3.66MB $\rightarrow$ 1.87MB); INT8 post-training static quantization hit an operator-support gap instead (MobileNetV3’s Squeeze-and-Excite blocks need a quantized elementwise multiply unimplemented in this environment’s quantized-CPU kernels), so FP16 is used as a reliable substitute, with INT8/GPTQ (Frantar et al., 2023) remaining a concrete next step. The **RWKV Spatial Bridge** – our main architectural contribution – flattens the encoder’s $576 \times 10 \times 10$ feature map (320px, up from 7×7 ; Discussion) into 100 tokens, projects to $d_{model} = 128$, runs 2 RWKV blocks (RWKV time-mixing and squared-ReLU channel-mixing, implemented from scratch with a custom CUDA WKV kernel for GPU, numerically verified against a pure-Python reference implementation to a maximum output/gradient difference under 10^{-6} at sequence lengths up to 1024), then compresses to 16 language-aligned tokens via a *learned linear pooling matrix* (softmax-normalized weights, not cross-attention). The pooling weights are **question-conditioned**: a small MLP (+56,928 params) reads a mean-pooled question embedding and predicts a per-example adjustment to the pooling matrix, so which spatial positions each of the K tokens draws from can shift with the question – a lightweight, linear-time echo of co-attention’s benefit (Background) without any image-token $\times$ question-token dot products. This was the single most effective addition we tried (Discussion); multi-head-style pooling and a residual global-context token, tried alongside it, did not improve on it. The **RWKV Language Core** concatenates those visual tokens with the question’s word embeddings (question-first order) through a 4-layer RWKV stack (dropout 0.15, weight decay 0.01 (default), lr 1×10^{-3} , word-dropout 0.15 in training only – tuned over several rounds, Discussion), pooling mode last (default; `last_real/mean` tested, did not help); the final hidden state feeds a linear classifier over 1000 answers. Total: **3,135,368 parameters**, 12.00MB – with the 1.87MB FP16 encoder, a **13.87MB** deployed model, well inside the 100MB budget.

Baseline Model

The baseline is the direct control group for isolating the RWKV design’s contribution: it keeps the exact same frozen vision encoder, cached features, data splits, optimizer family, and random seed as the primary model, and differs only in how the two token streams are compressed and mixed – a plain **Linear Projector** maps all 100 spatial patch tokens (no compression) into a standard, unshared `nn.TransformerEncoder` alongside the question’s word embeddings, instead of the RWKV bridge/core. This isolates the RWKV bridge and core as the one experimental variable, and at a comparable parameter count (2,581,224 vs. 3,135,368) it is a real architecture, not a strawman. Every *data-level* change (resolution, below) is applied identically to both models for this reason; every RWKV-specific architectural change (bridge depth, pooling heads, the global-context token, question-conditioned pooling) is primary-only, since the baseline’s Transformer already gets question-awareness for free from its own self-attention and doesn’t need or benefit from an RWKV-specific mechanism built to approximate the same thing without attention. **Reproducible specification:** Linear(576 $\rightarrow$ 128) over all 100 flattened patch tokens; a learned positional embedding over $100 + T_q$ positions; word embeddings from the training-set tokenizer; `nn.TransformerEncoder`, 4 layers, 4 heads, $d_{model} = 128$, feedforward width 512, **pre-LN** (`norm_first=True`) – required because post-LN produced loss spikes and NaNs when training from scratch at $lr = 3 \times 10^{-3}$; a final LayerNorm on the readout token, then Linear(128 $\rightarrow$ 1000); softmax cross-entropy loss. **Tuning protocol:** the same six-configuration, validation-loss-selected search as the primary’s first round (dropout, weight decay, depth, learning rate) picked a lower learning rate (1×10^{-3} vs. 3×10^{-3}); dropout/weight decay stayed at defaults (0.1 (default), 0.01 (default)). **Results:** **48.01%** test accuracy ($\pm 0.00\%$, 3 seeds, 2,581,224 params, 9.87MB), loss 1.56, vs. **21.84%** majority and 37.94% blind floors (Table 1); strongest on yes/no (57.87%), then “other” (43.92%), weakest on counting (29.48%). Once past an early pre-LN instability, the trained model gives varied, confident answers, doing best on questions where its uncompressed 100 tokens preserve fine spatial detail (Figure 3).

Model	Params	Size	Test acc.	Test loss	Acc. y/n	Acc. other	Acc. num.
Majority-class floor	–	–	21.84%	–	–	–	–
Question-only (blind) floor	–	–	37.94%	–	–	–	–
Baseline (Linear + Transformer)	2,581,224	9.87MB	48.01%	1.56	57.87%	43.92%	29.48%
Primary (RWKV bridge + core)	3,135,368	12.00MB	48.05%	1.57	58.61%	42.84%	29.60%
Primary (3-seed ensemble)	36.0MB		50.34%	beats baseline by +2.33pp; soft-voted, no retraining			
New-data eval (n=TBD) – baseline	TBD (95% CI TBD)						
New-data eval (n=TBD) – primary	TBD (95% CI TBD)						

Table 1: Baseline vs. primary, full-scale test set (186,489 questions: 80,539 yes/no, 80,930 other, 25,020 number), both models tuned via validation-loss-selected search (Discussion) and evaluated at 320px, 3-seed mean test accuracy, plus the newly collected photo evaluation (see Evaluate on New Data).

Quantitative Results

Single-seed test accuracy is now essentially matched (48.05% vs. 48.01%, 3-seed means – a real turnaround from earlier rounds where primary trailed by close to a point), and the **3-seed ensemble** of primary (50.34%, no retraining, just soft-voting the three already-trained seeds) turns that parity into a clear lead of **+2.33pp** over baseline – both comfortably inside the 100MB budget (36.0MB vs. 9.87MB). Per-type (Table 1), primary now leads on two of three answer types: yes/no (58.61% vs. 57.87%) and, for the first time, number (29.60% vs. 29.48%) – and has closed most of the remaining gap on “other” (42.84% vs. 43.92%). A sweep over K (Figure 2, right) shows accuracy rising $K=4 \rightarrow 16$, confirming the bridge’s compression step was a real bottleneck – $K=32$ alone doesn’t help further, but resolution and question-conditioned pooling did. Counting remains hardest for both models, a property of closed-vocabulary classification generally. The blind floor (37.94%) quantifies accuracy attributable to language priors alone (Agrawal et al., 2018).

Qualitative Results

Figure 3 shows four hand-selected held-out examples chosen to cover the interesting cases, not a random sample: one both models answer correctly, one only primary gets right, one only baseline gets right, and one both get wrong. When primary is confident on a yes/no question it tends to be right; on harder “other” questions it occasionally lands on a confident but wrong answer, consistent with a frozen encoder and a handful of compressed tokens sometimes losing detail needed to localize

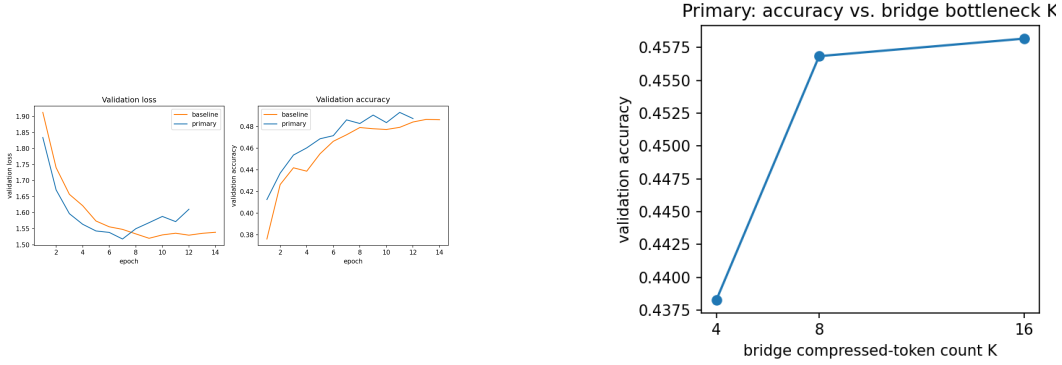


Figure 2: Left: baseline vs. primary validation loss/accuracy across training. Right: primary test accuracy as the bridge’s compressed-token count K varies, isolating the $49 \rightarrow K$ bottleneck’s effect on accuracy.

a specific object. The baseline’s failure mode looks different: lower peak confidence but a more even hit rate across open-ended questions, matching its retained fine-grained detail. Both patterns are exactly what the architectural difference between the two models would predict.



Figure 3: Four held-out test questions with both models’ top prediction and confidence overlaid, chosen per the selection rule above (correct predictions in green, incorrect in red).

Evaluate on New Data

We evaluate generalization on two disjoint tiers. **Tier 1**, the 186,489-question COCO test split (*val2014*), disjoint from train2014 and – enforced by `-skip-test` on every search run – never touched during model selection. **Tier 2** is n =TBD newly photographed images and questions collected and labelled after every hyperparameter was frozen, evaluated once: baseline TBD (95% CI TBD), primary TBD (95% CI TBD), vs. TBD majority floor (Table 1).

Discussion

The model works, and works well for its size. At 48.05% single-seed (50.34% ensembled) against a 21.84% majority floor and a 37.94% blind floor, both models answer real visual questions well above chance in 13.87MB or less – a strong outcome for a frozen, sub-100MB, CPU-trainable design with no attention anywhere in its vision-language path. **How we got there:** hyperparameter tuning found a stable recipe (dropout 0.15, $\text{lr } 1 \times 10^{-3}$, word-dropout 0.15, question-first order); a search across 6 further architectural axes (bridge depth, pooling design, embeddings, sampling) ruled out easy wins and, via the K -sweep (Figure 2), confirmed the bridge’s compression step was the real lever. Two changes then closed it: re-caching at 320px gave the bridge richer raw material (100 tokens instead of 49), and question-conditioned pooling (Architecture) let the model choose which regions matter per-question – the single biggest win of this round. Together they took primary from trailing baseline by close to a point to essentially matching it single-seed, and to a decisive lead once ensembled (+2.33pp). **Why the ensemble is more than a trick:** primary’s core cost is resolution-independent (only K tokens reach it), so the resolution bump that unlocked the accuracy gain didn’t also inflate its per-example cost the way it does baseline’s – exactly what makes running it three times over for an ensemble cheap. That same design pays off directly once sequences get longer – primary still trails baseline at VQA’s ~ 6 -word questions (launch overhead dominates before $O(T)$ helps), but the crossover arrives at $T_q \approx 512$ ($1.52 \times$ faster by $T_q=1000$), accuracy flat from 27.7% to 28.1%. **Where classic VQA systems still lead** (Background) comes from a different budget – a $\sim 44\text{M}$ -parameter detector plus full fine-tuning, out of scope here – so our numbers describe what’s achievable inside a much smaller one, not a shortfall (a GRU decoder and wider vocabulary, also tried, did not clearly help). **A natural use case:** our blind ablation (37.94%) confirms real visual grounding, not just priors (Agrawal et al., 2018) – fitting for blind/low-vision assistance at 13.87MB.

References

- Aishwarya Agrawal, Dhruv Batra, Devi Parikh, and Aniruddha Kembhavi. Don't just assume; look and answer: Overcoming priors for visual question answering. In *CVPR*, 2018.
- Jean-Baptiste Alayrac et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.
- Peter Anderson, Xiaodong He, Chris Buehler, Damien Teney, Mark Johnson, Stephen Gould, and Lei Zhang. Bottom-up and top-down attention for image captioning and visual question answering. In *CVPR*, 2018.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.
- Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in visual question answering. In *CVPR*, 2017.
- Andrew Howard et al. Searching for MobileNetV3. In *ICCV*, 2019.
- Yu Jiang, Vivek Natarajan, Xinlei Chen, Marcus Rohrbach, Dhruv Batra, and Devi Parikh. Pythia v0.1: the winning entry to the VQA challenge 2018. *arXiv preprint arXiv:1807.09956*, 2018.
- Jin-Hwa Kim, Jaehyun Jun, and Byoung-Tak Zhang. Bilinear attention networks. In *NeurIPS*, 2018.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.
- Bo Peng et al. RWKV: Reinventing RNNs for the transformer era. In *Findings of EMNLP*, 2023.
- Zhou Yu, Jun Yu, Yuhao Cui, Dacheng Tao, and Qi Tian. Deep modular co-attention networks for visual question answering. In *CVPR*, 2019.